Nile University

3D Solar System Visualization

Using Three.js

Phase One

CSCI452

Members:

Sherin Shady - 221000008

Jana Ahmed - 221000003

Malak Mohamed – 221000447

1. Project Description and Objective

- This project is an interactive 3D Solar System simulation created using Three.js. The main goal of the project is to visualize the planets, their orbits, and their rotations in a realistic and engaging way. The user can move around the scene, zoom in and out, click on planets to view labels, and even toggle their appearance. The project aims to give the viewer a simple, friendly, and interactive experience of exploring space.

The scene includes:

- A 3D sun with glow effects
- All major planets from Mercury to Pluto
- Each planet has its own texture, size, distance, and orbit speed
- Rings for Saturn and Uranus
- Orbit rings drawn around each planet
- A starry background
- Interactive features like:
  - Clicking a planet to show a label
  - Changing the planet's texture on click
  - Double-clicking to hide/show any object
  - WASD movement + OrbitControls (mouse movement)

The objective of the project is:

- To build a complete 3D environment
- To create realistic motion for planets around the sun
- To allow users to interact with the objects in a simple way
- To practice using Three.js features such as lighting, textures, raycasting, camera controls, and animations

2. Sequence of Steps Followed to Accomplish the Project

Below is the workflow of how the project is built from start to finish:

Step 1: Setting up the Scene

- Create the main Three.js scene, camera, and renderer.
- Enable shadows and set the background color.
- Add ambient light and a point light representing the sun.

Step 2: Loading Textures

- Use THREE.TextureLoader() to load planet textures (Earth, Mars, Jupiter, etc.) and the space background image.
- Apply the starry background to the entire scene.

Step 3: Creating the Sun

- Build a sphere geometry for the sun.
- Add a glowing effect around the sun using a canvas-based sprite.
- Position a point light at the sun so planets receive realistic lighting.

Step 4: Creating Planets

For each planet:

- Create a sphere with the correct size.
- Apply its texture.
- Create a pivot (invisible object) that controls its orbit.
- Position the planet at its distance from the sun.
- Add an orbit ring to visualize the path.
- This step is repeated for all planets from Mercury to Pluto.

Step 5: Adding Special Features

- Add Saturn's rings using a RingGeometry with texture.
- Add Uranus's rings.
- Store all planets in arrays to make animation and interaction easier.

Step 6: Adding Interactivity

- Enable OrbitControls to allow camera movement with the mouse.
- Use a Raycaster to detect clicks on planets.
- When a planet is clicked:
  - Show a label with its name, radius, and distance.
  - Toggle its texture (texture ↔ grayscale color).
  - Double-clicking hides or shows the selected object.
  - Add WASD keyboard controls to move the camera forward/backward/left/right/up/down.

Step 7: Animation Loop

Inside the animate() function:

- Rotate each planet around the sun by rotating its pivot.
- Rotate planets around their own axis.
- Rotate the sun slightly.
- Update camera controls.
- Render the scene every frame.

Step 8: Handling Screen Resizing

- Update camera aspect ratio and renderer size when the browser window is resized.